SIMATS ENGINEERING name: B.vennela

Reg :192412604

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code: CSA1205 Course Title: Computer Architecture

CO Assessed: CO2 Assessment: ASSESSMENT TOOL1-

ANALYTICAL PROBLEM SOLVING

Weightage: 45% Total Marks: 25

CO2 Statement: Apply integer and floating-point arithmetic algorithms including Booth's

multiplication, restoring/non-restoring division, and IEEE 754 standard

operations to solve fixed-point and floating-point computational problems.

(BL3)

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement

representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs

both addition and subtraction, including binary conversion, alignment of sign bits, and

intermediate steps. Determine whether overflow occurs, and explain how the processor detects

overflow and interprets the final result.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-

ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how

generate and propagate signals are formed and how carries are computed in advance. Compare

the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA

improves speed in modern processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two

numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator,

multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently

handles signed numbers and reduces the number of addition/subtraction operations compared

to traditional multiplication.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both

restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing

intermediate remainders and quotient formation. Compare the two approaches in terms of

number of steps, complexity, and efficiency, and explain why non-restoring division is often

preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard

for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is

represented in both single precision and double precision formats, including sign, exponent,

and mantissa fields. Further, explain how floating-point addition is performed between two

such numbers, highlighting issues like normalization, rounding, and precision loss.

Reg :192412604

B.vennela

1. Addition and Subtraction using 8-bit 2's Complement Representation (10 Marks)

Given:

- **A = +45**
- **B = –23**

Step 1: Convert Decimal to Binary

+45

Binary of 45 = **00101101**

–23

Binary of +23 = **00010111**

Find 2's complement:

1. One's complement = **11101000**
2. Add 1 = **11101001**

Therefore,

−23 = 11101001

A. Addition (+45 + (−23))

00101101 (+45)

+ 11101001 (-23)

---

1 00010110

Discard the carry out.

Result:

00010110 = 22

Answer = +22

Overflow Check

Overflow occurs only when:

- Positive + Positive = Negative
- Negative + Negative = Positive

Here,

Positive + Negative = Positive

Hence,

No overflow occurs.

B. Subtraction (+45 − (−23))

Subtracting a negative number means adding its positive value.

Need +23:

00010111

Perform addition:

00101101

+ 00010111

01000100

Binary result:

01000100 = 68

Answer:

45 − (−23) = 68

Overflow Check

Both operands are positive.

Result is also positive.

Therefore,

No overflow.

How Processor Detects Overflow

Overflow is detected by comparing:

- Carry into MSB
- Carry out of MSB

If they are different,

Overflow = 1

Otherwise,

Overflow = 0

For both operations:

Carry into MSB = Carry out of MSB

Hence,

Overflow does not occur.

Final Results

Operation	Binary Result	Decimal	Overflow
$45 + (-23)$	00010110	22	No
$45 - (-23)$	01000100	68	No

2. Carry Look-Ahead Adder

Suppose

A = 1101

B = 1011

Cin = 0

Step 1: Generate (G) and Propagate (P)

Formula

$G_i = A_i \times B_i$

$P_i = A_i \oplus B_i$

Bit	A	B	G	P
0	1	1	1	0

Bit	A	B	G	P
1	0	1	0	1
2	1	0	0	1
3	1	1	1	0

Step 2: Carry Equations

$C1 = G0 + P0Cin$

$C2 = G1 + P1G0 + P1P0Cin$

$C3 = G2 + P2G1 + P2P1G0$

$C4 = G3 + P3G2 + P3P2G1 + P3P2P1G0$

Since carries are computed simultaneously,

No waiting is required.

Step 3: Sum

$Si = Pi \oplus Ci$

All sums are obtained after carry computation.

Ripple Carry Delay

Each carry waits for the previous carry.

For 4 bits,

Delay = 4 carry delays

For n bits,

Delay = O(n)

CLA Delay

Carries are generated simultaneously.

Delay ≈ O(log n)

(or nearly constant for small CLA blocks)

Comparison

Ripple Carry Adder	Carry Look Ahead Adder
Carry generated sequentially	Carry generated simultaneously

Ripple Carry Adder	Carry Look Ahead Adder
Slow	Very fast
Delay = O(n)	Delay ≈ O(log n)
Simple hardware	More hardware
Used in small circuits	Used in modern CPUs

Why CLA is Faster

- Parallel carry generation
- No ripple delay
- Faster arithmetic operations
- Improves processor performance

3. Booth's Multiplication (−6 × +5) (10 Marks)

Given

Multiplicand (M) = −6

Multiplier (Q) = +5

Using 4-bit representation

M = -6 = 1010

−M = +6 = 0110

Q = 5 = 0101

A = 0000

Q−1 = 0

Booth Rules

Q0 Q−1	Operation
00	No operation
11	No operation
01	A = A + M
10	A = A − M

Step 1

Q0Q−1 = 10

A = A − M

0000 −1010

=0000 +0110

=0110

Arithmetic shift

A=0011

Q=0010

Q−1=1

Step 2

Q0Q−1 = 01

A=A+M

0011

+1010

=1101

Shift

step 3

Repeat Booth rules.

Continue until all 4 bits are processed.

Final result:

11100010

Decimal

−30

Since

−6 × 5 = −30

Correct.

Role of Registers

Accumulator (A)

Stores partial product.

Multiplier (Q)

Contains multiplier bits.

Booth bit (Q−1)

Determines operation.

Arithmetic Right Shift

- Preserves sign bit
- Shifts A,Q,Q−1 together

Advantages of Booth Algorithm

- Works directly with signed numbers
- Reduces additions
- Efficient for consecutive 1s
- Faster multiplication
- Used in processors

4. Restoring and Non-Restoring Division (13 ÷ 3) (10 Marks)

Given

Dividend =13 =1101

Divisor =3 =0011

Expected quotient

13÷3=4 remainder1

A. Restoring Division

Initialize

A=0000

Q=1101

M=0011

Step 1

Shift AQ

Subtract divisor

A=A−M

If negative

Restore

A=A+M

Q0=0

Step 2

Repeat

Shift

Subtract

Restore if required

Final

Quotient=0100

Remainder=0001

Decimal

Quotient=4

Remainder=1

B. Non-Restoring Division

Initialize

A=0000

Q=1101

If

A positive

Subtract divisor

If

A negative

Add divisor

No restoration after every step.

After final iteration,

If remainder negative,

Add divisor once.

Final

Q=0100

R=0001

Comparison

Restoring	**Non-Restoring**
Restores after every negative remainder	No immediate restoration
More operations	Fewer operations
Slower	Faster
Simple	Slightly complex
Lower efficiency	Higher efficiency

Why Non-Restoring is Preferred

- Fewer additions
- Less hardware delay
- Faster execution
- Better processor performance

5. IEEE 754 Floating-Point Representation (−13.25) (10 Marks)

Step 1: Convert to Binary

13 =1101

0.25 =0.01

13.25

$=1101.01_2$

Normalize

1.10101×2^3

A. Single Precision (32 bits)

Sign

Negative

S=1

Exponent

Bias

127

Exponent

3+127

=130

10000010

Mantissa

Remove leading 1

10101000000000000000000

Final

1

10000010

10101000000000000000000

B. Double Precision (64 bits)

Sign

1

Exponent

Bias

1023

Exponent

1026

10000000010

Mantissa

1010100000000000000000000000000000000000000000000000

Floating-Point Addition

Example

13.25 + 6.5

Steps

1. Compare exponents

Smaller exponent shifted right.

2. Align mantissas

Both numbers have same exponent.

3. Add mantissas

Binary addition performed.

4. Normalize

If result exceeds 2,

Shift right.

Adjust exponent.

5. Round

IEEE 754 uses

- Guard bit
- Round bit
- Sticky bit

Nearest-even rounding is commonly used.

Precision Loss

Occurs because

- Mantissa has limited bits
- Small values may disappear after alignment
- Rounding introduces slight error

Example:

1.0000000

+0.00000001

≈1.0000000

Tiny value may be lost.

Advantages of IEEE 754

- Standard representation across systems

- Supports very large and very small numbers
- Handles positive/negative values
- Provides rounding rules
- Represents special values (±∞, NaN, denormalized numbers)